P3034R1
Module Declarations Shouldn’t be Macros

Published Proposal, 2024-03-21

This version:
http://wg21.link/P3034R1

Author:
Michael Spencer (Apple)

Audience:
SG15

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Under the current standard, determining which source file defines a given named module requires pre-processing. This increases the latency of builds unless additional restrictions are imposed.

Table of Contents

§ 1. Effects of This Paper

This paper makes the following ill-formed by forbidding macro expansion in the name of module declarations.

version.h:

```
#ifndef VERSION_H
#define VERSION_H

#define VERSION libv5

#endif
```

lib.cppm:

```
module;
#include "version.h"
export module VERSION;
```

This is still valid in `import` declarations, as are macros in the attribute following a module declaration.

§ 2. The Issue

Given `import creature;`, the implementation needs to know which TU contains `export module creature;`. There are many possible ways to do this, but the current specification makes this diffcult in the general case.

```
module;
#include <ponies.h>
export module creature;
// ...
```

In this example the implementation must either have an oracle, or preprocess up until the `export module creature;` preprocessing directive to determine which module this TU defines, as the *pp-tokens* that make up the module name are themselves subject to macro replacement cpp.module/2, including any macros brought in by `#include <ponies.h>`.

This means that build systems must either:

- Do preprocessing up front to determine where modules are defined, adding latency to the build
- Require explicit metadata for modules, even local to the project
- Require module names to match file names
- Not support such cases

§ 2.1. Sketch of a Simple Build System

For a more concrete example of where this becomes a problem, here's a sketch of a simple build system using ninja.

As input you have 100 `*.cpp` and `*.cppm` files where `*.cppm` files are importable TUs, and a `build.ninja` file with rules for building each TU, but without module dependencies.

If you started a build with `-j16`, 16 of those TUs would start building, and start hitting `import`s which need to be resolved. However, there are still 84 TUs that haven't started building yet that likely contain the module declarations to resolve these imports.

If we want as close to a zero-configuration build system as possible without also adding restrictions on module names, we must add a module discovery phase that runs before the first dependent `import` is resolved. This can either be explicit in the build system, or part of the module mapper. Currently this discovery phase is required to do preprocessing which adds a delay before any real compilation can begin.

§ 2.2. Caching Build Systems

Another case where latency is particularly important is in caching build systems. Let's assume the same collection of 100 TUs as before, but this time our build system can return cached results for compilations. In order to do this in a reproducable manner the cache key must be dependent on the the full input to each compilation, including all source files and modules it depends on, including how they are built, recursively.

In a non-modules world this can be computed by minimal preprocessing; however, while resolving `import`s to module declarations is not needed for discovering direct dependencies, it is needed to determine the cache key for a compilation. Latency is important here because time spent discovering module declarations delays time to first byte for any cache hits.

§ 3. Module Declaration Discovery

Due to the structure of a *preprocessing-file*, the *pp-module* line is discoverable at the start of phase 4 of translation without processing any `#include`s or resolving any preprocessing conditionals. For some environments this can be done without a command line at all, or with only a partial one. The only thing preventing this is that the *module-name* and *module-partition* tokens may be subject to macro replacement.

If this were not the case, then a reasonably simple parser can determine the *module-name* and *module-partition* of a source file without calling out to compiler specific tooling.

§ 4. Compatibility

This is a breaking change with C++20 and C++23, however, given the limited current deployment of modules and rarity of such use cases, the breakage is expected to be minimal.

§ 5. Wording

Apply the following wording as a DR:

Modify Module directive [**cpp.module**] inserting a paragraph after paragraph 1 as follows:

pp-module:

`export`$_{opt}$ `module` *pp-tokens*$_{opt}$ `;` *new-line*

1 A *pp-module* shall not appear in a context where `module` or (if it is the first token of the *pp-module*) `export` is an identifier defined as an object-like macro.

2 The *pp-tokens*, if any, of a *pp-module* shall be of the form:

pp-module-name pp-module-partition$_{opt}$ *pp-tokens*$_{opt}$

where the *pp-tokens* (if any) shall not begin with a `(` preprocessing token and the grammar non-terminals are defined as:

pp-module-name:

pp-module-name-qualifier$_{opt}$ *identifier*

pp-module-partition:

`:` *pp-module-name-qualifier*$_{opt}$ *identifier*

pp-module-name-qualifier:

identifier `.`

pp-module-name-qualifier identifier `.`

No *identifier* in the *pp-module-name* or *pp-module-partition* shall currently be defined as an object-like macro.